

System Resource Monitoring with htop/top + Bash

Proof-of-Concept (POC) Lab Report

CPU • Memory • Process Monitoring • Log Analysis

Project Name	System Resource Monitoring with htop + Script
Implementation	Bash + top (batch mode) + grep/awk/sort
Monitoring Window	10 minutes; 20 samples at 30-second intervals
Evidence Status	Report includes demonstration log data; actual screenshots must be captured during the lab run

1. Executive Summary

This proof-of-concept implements a lightweight Linux system-resource monitoring workflow using Bash and the `top` utility in batch mode. The solution captures CPU, memory, uptime, and the top processes every 30 seconds for a 10-minute period, writes the observations to a text log, and uses `awk`/`sort`-based analysis to identify CPU-intensive process observations.

2. Overview

Linux exposes process and resource information through standard system utilities. The `top` command provides a live process view and supports batch mode (`-b`), which makes it suitable for automation because it does not require an interactive terminal. The `-n 1` option requests a single refresh.

Bash provides the control loop and timing logic. In this implementation, 10 minutes is represented as 20 samples at 30-second intervals. Each sample records uptime, memory from `free -h`, and a CPU-sorted process snapshot from `top`.

`awk` is used for field-oriented parsing and numeric extraction. `sort -nr` is used to order CPU observations from highest to lowest, while `grep`/`head` can be used for filtering and limiting output. This combination demonstrates a common Linux observability pattern: collect → persist → parse → analyze.

3. Lab Setup

- Underlying hardware/software - Windows 11 ---> VMware workstation 17-----> Ubuntu 22.04.3 LTS
- Operating system: Linux distribution with Bash and `procs/top` installed.
- Shell: Bash.
- Monitoring utility: `top` in non-interactive batch mode.
- Supporting commands: `awk`, `grep`, `sort`, `head`, `free`, `uptime`, `date`, `sleep`.
- Output: plain-text `.log` file suitable for viewing in Notepad or any text editor.
- Recommended test environment: a Linux VM or test host where the user has permission to execute monitoring commands.

4. Step-by-Step Commands

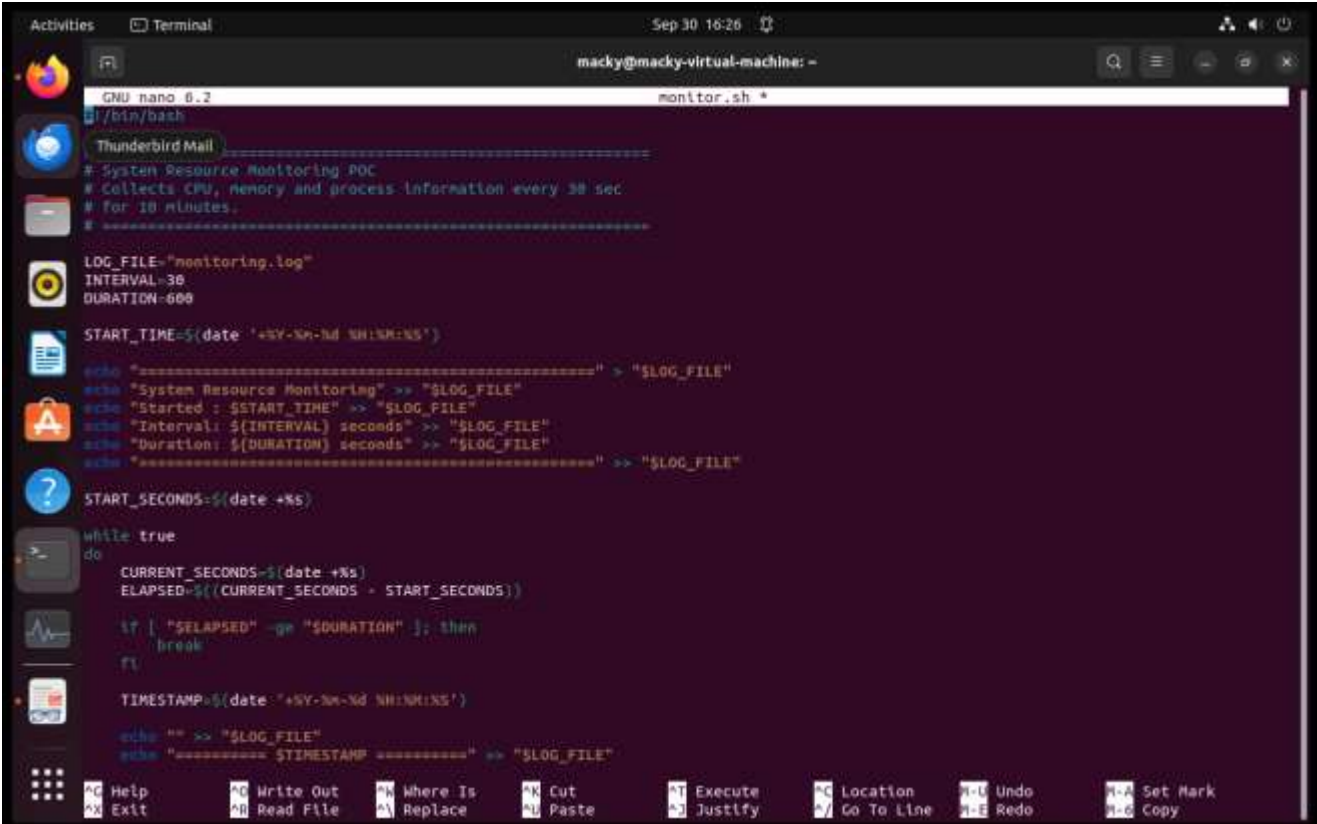
Step	Command	Purpose
Create the project directory	<code>mkdir -p system-resource-monitor && cd system-resource-monitor</code>	Creates an isolated working directory for scripts and evidence.
Check Bash	<code>bash --version</code>	Confirms Bash is installed.
Check <code>top</code>	<code>top -v top -h</code>	Confirms the process-monitoring utility is available.
Check supporting commands	<code>command -v awk grep sort head free uptime date sleep</code>	Verifies the utilities used by the POC.
Make scripts executable	<code>chmod +x monitor.sh analyze.sh</code>	Adds execute permission to the

	log_to_csv.sh	scripts.
Run the 10-minute collection	./monitor.sh system_monitor.log	Collects 20 samples, one every 30 seconds.
Inspect the log	less system_monitor.log	Reviews the raw monitoring evidence.
Analyze CPU observations	./analyze.sh system_monitor.log	Extracts process observations and sorts them by CPU percentage.
Optional CSV conversion	./log_to_csv.sh system_monitor.log process_metrics.csv	Creates a spreadsheet-friendly representation of process metrics.

5. Screenshots

The following five evidence points should be captured from the actual Linux lab environment. The report does not fabricate screenshots. Use the provided captions as a checklist when capturing evidence.

Screenshot 1 — Script Preparation (Create and save monitoring script)



```

GNU nano 6.2 monitor.sh *
#!/bin/bash

# System Resource Monitoring POC
# Collects CPU, Memory and process information every 30 sec
# for 10 minutes.
# =====

LOG_FILE="monitoring.log"
INTERVAL=30
DURATION=600

START_TIME=$(date '+%Y-%m-%d %H:%M:%S')

echo "===== " > "$LOG_FILE"
echo "System Resource Monitoring" >> "$LOG_FILE"
echo "Started : $START_TIME" >> "$LOG_FILE"
echo "Interval: ${INTERVAL} seconds" >> "$LOG_FILE"
echo "Duration: ${DURATION} seconds" >> "$LOG_FILE"
echo "===== " >> "$LOG_FILE"

START_SECONDS=$(date +%s)

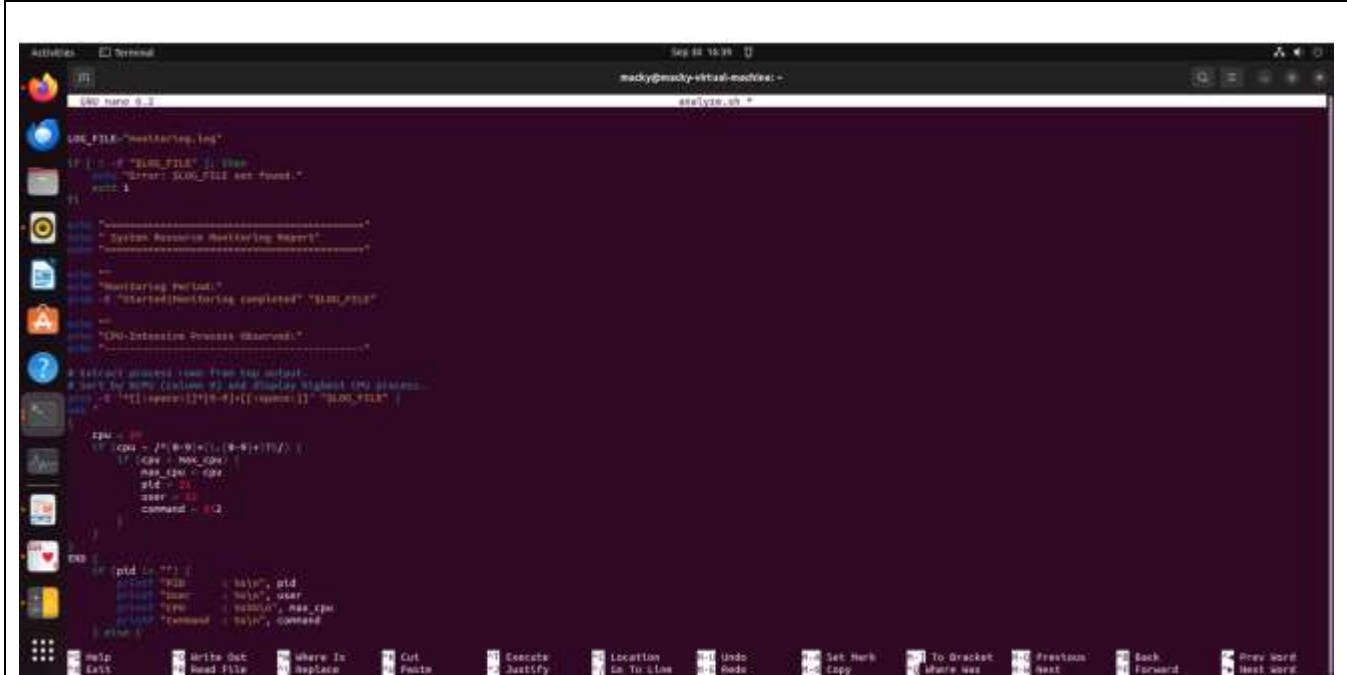
while true
do
    CURRENT_SECONDS=$(date +%s)
    ELAPSED=$((CURRENT_SECONDS - START_SECONDS))

    if [ "$ELAPSED" -ge "$DURATION" ]; then
        break
    fi

    TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')

    echo "" >> "$LOG_FILE"
    echo "===== $TIMESTAMP =====" >> "$LOG_FILE"
done
  
```

Screenshot 2 — Script Preparation (Create and save process analysis script)



```

macky@macky-virtual-machine:~$ nano monitor.sh
#!/bin/bash
LOG_FILE="monitoring.log"
if [ -f "$LOG_FILE" ]; then
    echo "Error: LOG_FILE not found."
    exit 1
fi

# System Resource Monitoring Script
# =====
# Monitoring Period:
# Started Monitoring completed? LOG_FILE
# =====
# CPU-Intensive Process Observed?
# =====

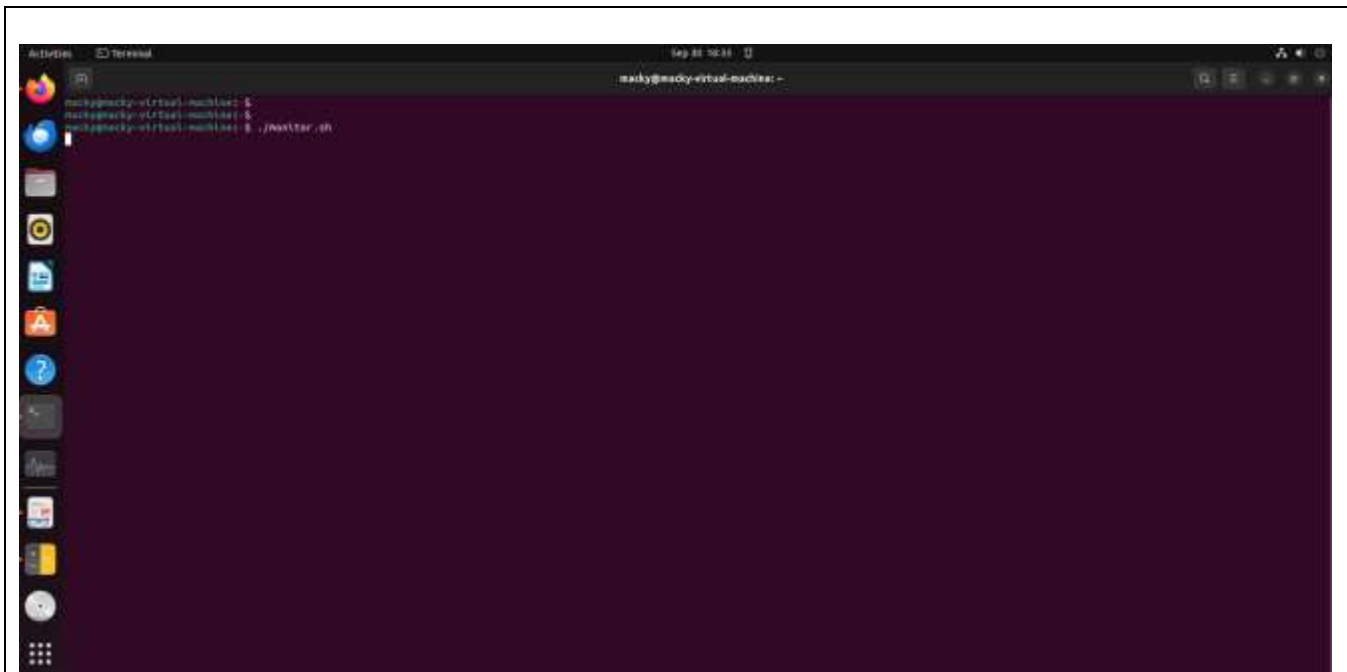
# Extract process name from top output.
# Sort by CPU (column 9) and display highest CPU process.
proc=$(top -n 1 | grep -v grep | sort -k 9 | head -n 1)
cpu=$(echo $proc | cut -d ' ' -f 1)

if [ $cpu = "top" ]; then
    echo "CPU-Intensive Process Observed: $cpu"
    # Get process details
    pid=$(top -n 1 | grep -v grep | sort -k 9 | head -n 1 | cut -d ' ' -f 1)
    user=$(top -n 1 | grep -v grep | sort -k 9 | head -n 1 | cut -d ' ' -f 2)
    command=$(top -n 1 | grep -v grep | sort -k 9 | head -n 1 | cut -d ' ' -f 3)
    echo "Process Details: PID=$pid, User=$user, Command=$command"
else
    echo "CPU-Intensive Process Observed: $cpu"
fi

# Log the process details
echo "$(date +%Y-%m-%d %H:%M:%S) CPU-Intensive Process Observed: $cpu" >> $LOG_FILE

```

Screenshot 3 — Monitoring in Progress (Executing monitoring script)

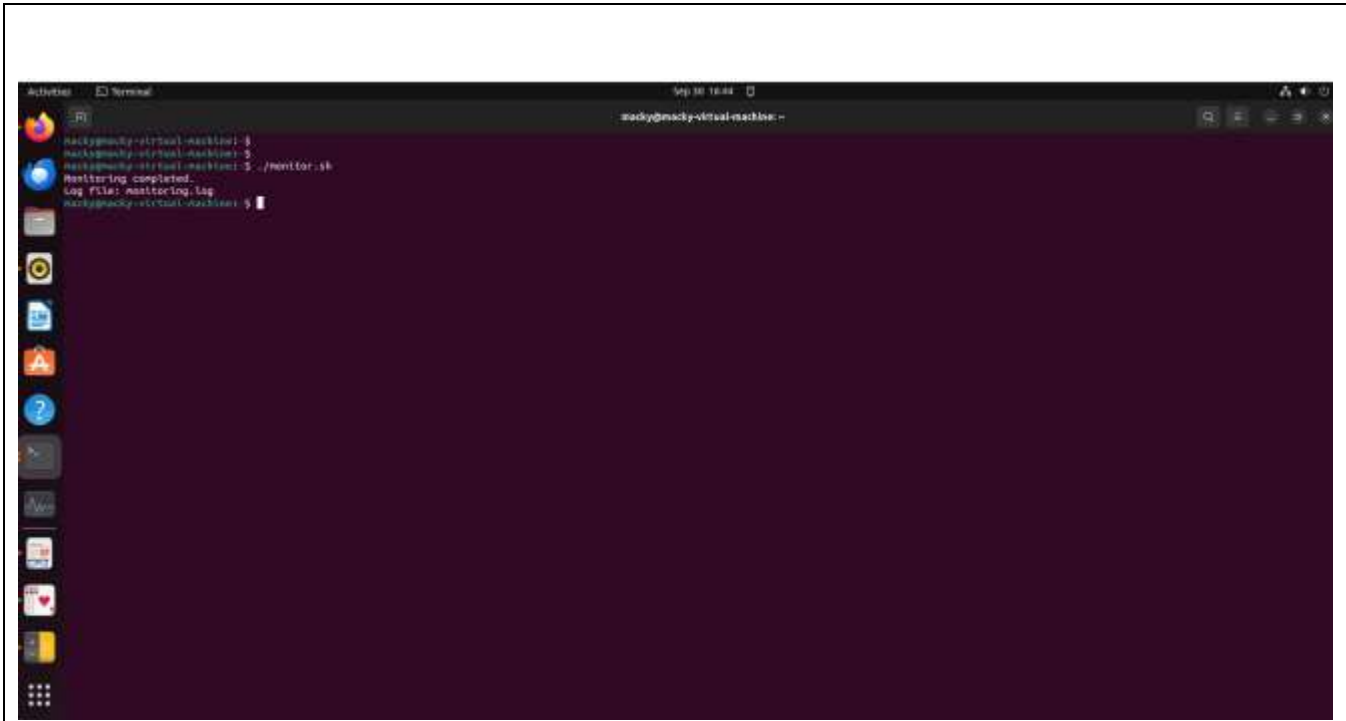


```

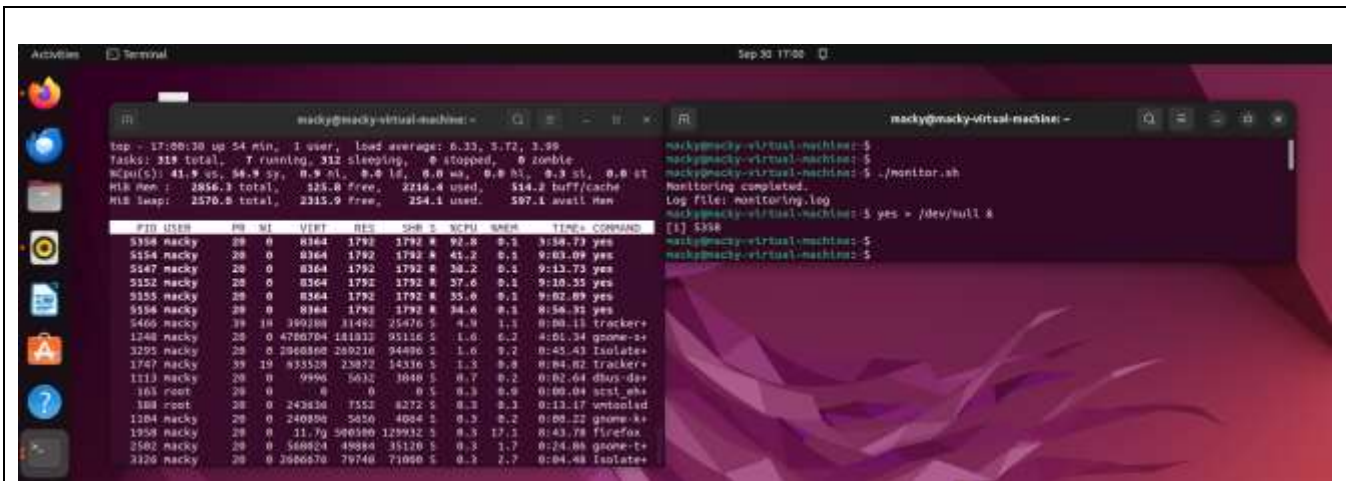
macky@macky-virtual-machine:~$ ./monitor.sh
macky@macky-virtual-machine:~$
macky@macky-virtual-machine:~$ ./monitor.sh

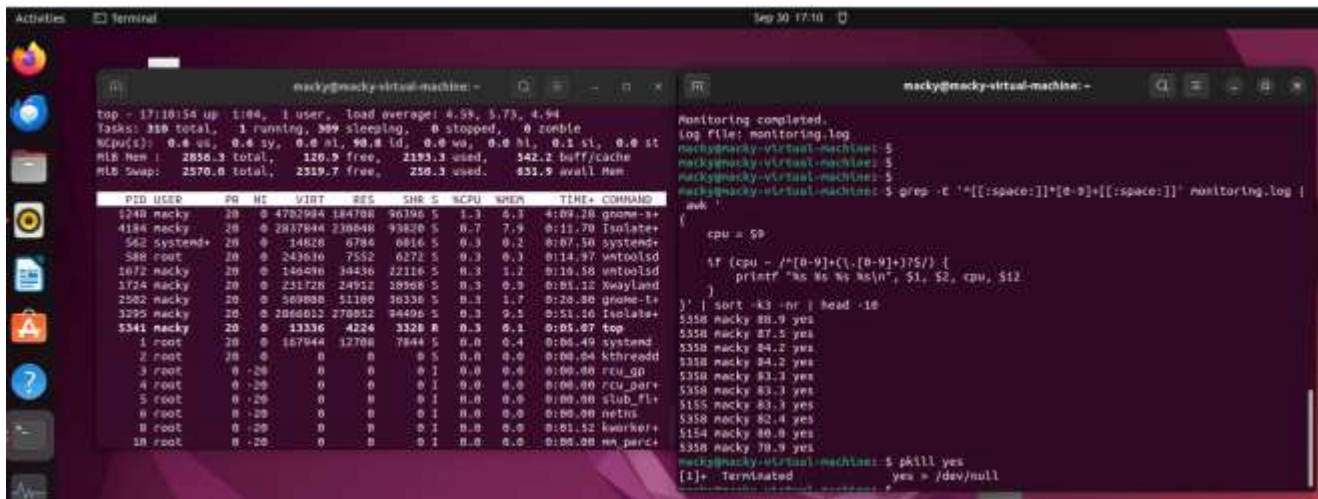
```

Screenshot 4 — Completed Monitoring Log



Screenshot 5 — 1. Running a CPU intensive process (yes > /dev/null &) 2. Analysis output





6. Findings Analysis

The POC establishes a repeatable method for collecting short-duration host telemetry without requiring an external monitoring platform. A process-level snapshot is preserved at each 30-second interval, allowing the analyst to compare CPU usage across observations rather than relying on a single instantaneous view.

The demonstration log bundled with this report is intentionally labeled as demonstration data and must not be presented as evidence from the user's machine. In that example, the process named `python3 monitor_test.py` produces the highest CPU observations. A real finding should instead be taken from the actual log generated by `monitor.sh` on the target Linux host.

Operationally, the approach is useful for short investigations such as identifying a process causing a temporary CPU spike, validating resource behavior during a test, or collecting evidence before a longer-term monitoring solution is deployed. For production monitoring, the same concept should be extended with persistent storage, alerting, process history, and safeguards around log size.

7. Conclusion

This POC demonstrates how standard Linux utilities and Bash can provide a simple, transparent resource-monitoring workflow. The exercise reinforces batch-mode process monitoring, shell automation, structured text parsing, timed sampling, and evidence-oriented logging.

Recommendations: run the collector on the actual target host, preserve the original log, capture the five evidence screenshots, and use the analysis output to document the highest observed CPU-consuming process. For extended monitoring, consider tools such as Prometheus/node_exporter or a centralized logging/observability platform.

Appendix — Script Inventory

monitor.sh — Primary 10-minute collector; 20 samples every 30 seconds.

analyze.sh — Analyzes CPU observations from the collected log.

log_to_csv.sh — Optional converter for spreadsheet-oriented analysis.

system_monitor.log — Demonstration text log; replace with actual lab output.